

FDA Pharmacovigilance Platform on Databricks

One Silver layer · two Gold consumers (RAG + Analytics) · Unity Catalog governance

notebook

Imperative reference implementation
(notebooks/)

DLT

Declarative pipeline equivalent
(pipelines/)

@dlt.expect

Data quality
gate

THE BUSINESS PROBLEM

Two consumer groups, same data, parallel pipelines — until now.

Pharmacovigilance teams spend hours per safety inquiry searching FDA labels and adverse-event reports. Epidemiologists need slice-and-dice analytics over the same data. Both consumers historically built parallel, divergent pipelines off the same source.

✗ The obvious approach

- Two parallel pipelines, one per consumer team
- Business definitions duplicated → analysts argue about whose count is right
- Schema changes happen twice
- Two sets of code to maintain
- Two access models to govern separately

✓ This project's load-bearing decision

- **One Silver layer** — business logic defined exactly once
- **Two Gold projections** — RAG chunks + star schema, both derived from Silver
- Same numbers everywhere; analysts stop arguing
- Schema changes in one place, propagate to both
- Unity Catalog governs everything under one access model

Why this design matters: adding a third consumer (e.g., a regulatory affairs export or a clinical trial enrollment dashboard) is a *new Gold projection*, not a new pipeline. The platform scales horizontally by consumer without re-doing the ingestion work.

JUMP TO SECTION

- 0

Problem Statement — why this exists→
- 1

Architecture Overview — medallion + serving→
- 2

DLT Deployment Topology — 4 pipelines→
- 3

Notebooks vs Declarative — side by side→
- 4

Event-Driven Orchestration — polling + cascade→
- 5

Governance — RLS & PII — multi-tenant→
- 6

Asset Bundle Deployment — dev → prod→
- 7

Implementation Playbook — end-to-end walkthrough→

DATA SOURCES

openFDA Drug Labels API

api.fda.gov/drug/label.json

- Structured prescribing info per drug
- Warnings · contraindications · interactions

openFDA Adverse Events API

api.fda.gov/drug/event.json

- Reported reactions per safety report
- Patient demographics · severity flag



HTTP fetch · retry + backoff

BRONZE · RAW & IMMUTABLE**fda_rag.bronze.openfda_raw**

drug · source · payload (JSON) · ingested_at

- **Append-only** — replay-safe, system-of-record
- Schema preserved as JSON string — survives upstream changes
- ~1,600 records (~800 labels + ~800 events) across 8 drugs

01_ingest_bronze.py.ipynb**pipelines/bronze/ingest_openfda.py**

@dlt.expect: payload_non_empty · source_valid

Parse · dedupe · type-cast

SILVER · CONFORMED ENTITIES**silver.drug_labels**

dedupe: drug_generic + manufacturer

- 10 typed columns extracted from JSON
- warnings · contraindications · dosage · boxed_warning · ...

@dlt.expect_or_drop: drug_generic_present

silver.adverse_events

dedupe: safetyreportid

- One row per FDA safety report
- reactions array · patient_age · patient_sex · serious · receive_date

@dlt.expect_or_drop: safetyreportid_present · reactions_non_empty

02a_silver.py.ipynb**pipelines/silver/conform_entities.py**

↑ Single source of truth — business logic defined exactly once

GOLD · CONSUMER-SHAPED PROJECTIONS**RAG PATH****gold.fda_chunks**

chunk_id (SHA-256) · drug · section · text

- **Section-aware chunking** (warnings, contraindications...)
- delta.enableChangeDataFeed = true
- +18% faithfulness vs naive token splits

02b_gold_rag.py.ipynb**pipelines/gold_rag/build_chunks.py**

@dlt.expect_or_drop: chunk_id · text_non_trivial

ANALYTICS PATH**Star schema · 4 dims + 1 fact**

SHA-256 surrogate keys

- **dim_drug** · **dim_patient** (age-banded) · **dim_reaction** (body-system) · **dim_date**
- **fact_adverse_event** — partitioned by date_key

02c_gold_analytics.py.ipynb**pipelines/gold_analytics/build_star_schema.py**

@dlt.expect_or_fail: surrogate keys present

Two serving paths

SERVING (IMPERATIVE — STAYS AS NOTEBOOKS)**Vector Search Index**

fda_chunks_index · TRIGGERED sync

- Embedding: databricks-gte-large-en

03_vector_index**LangChain RAG Chain**

retriever(k=5) → prompt → LLM → parser

Databricks SQL Warehouse

Serverless · parameterized dashboard

- 4 widgets: top reactions, demographic slice, quarterly trend, heatmap
- Sub-second on indexed star schema



- LLM: databricks-gpt-5-5-pro (temp 0.1)
- System prompt: cite, refuse OOC, disclaimer
- Registered: fda_rag.gold.fda_assistant

04_rag_chain**Model Serving Endpoint**

fda-rag-endpoint · scale-to-zero

- auto_capture_config → inference table

06_deploy_serving**FastAPI Gateway**

X-API-Key auth · request_id · structured logs

CONSUMERS**Pharmacovigilance team**

- Natural-language Q&A grounded in FDA data
- Every claim cited with [drug — section]
- 94% refusal rate on out-of-corpus questions

Epidemiologists

- Slice-and-dice across drug · reaction · age · time
- Top reactions, demographic breakdowns, trends
- Sub-second response on parameterized filters

CROSS-CUTTING**Unity Catalog**

- Lineage · access control · audit · model registry

MLflow

- Experiments · versioning · registered models · evals

Eval Harness

- Faithfulness 0.87 · Relevance 0.91 · p95 2.4s

05_evaluate**Ops Monitoring**

- Inference table view: latency, errors, refusal rate

07_monitoring**Tenant RLS & PII Mask**

- Row filter + column mask on Silver / Gold

08_setup_rls**Asset Bundle Deployment**

- databricks.yml + resources/ for dev → prod

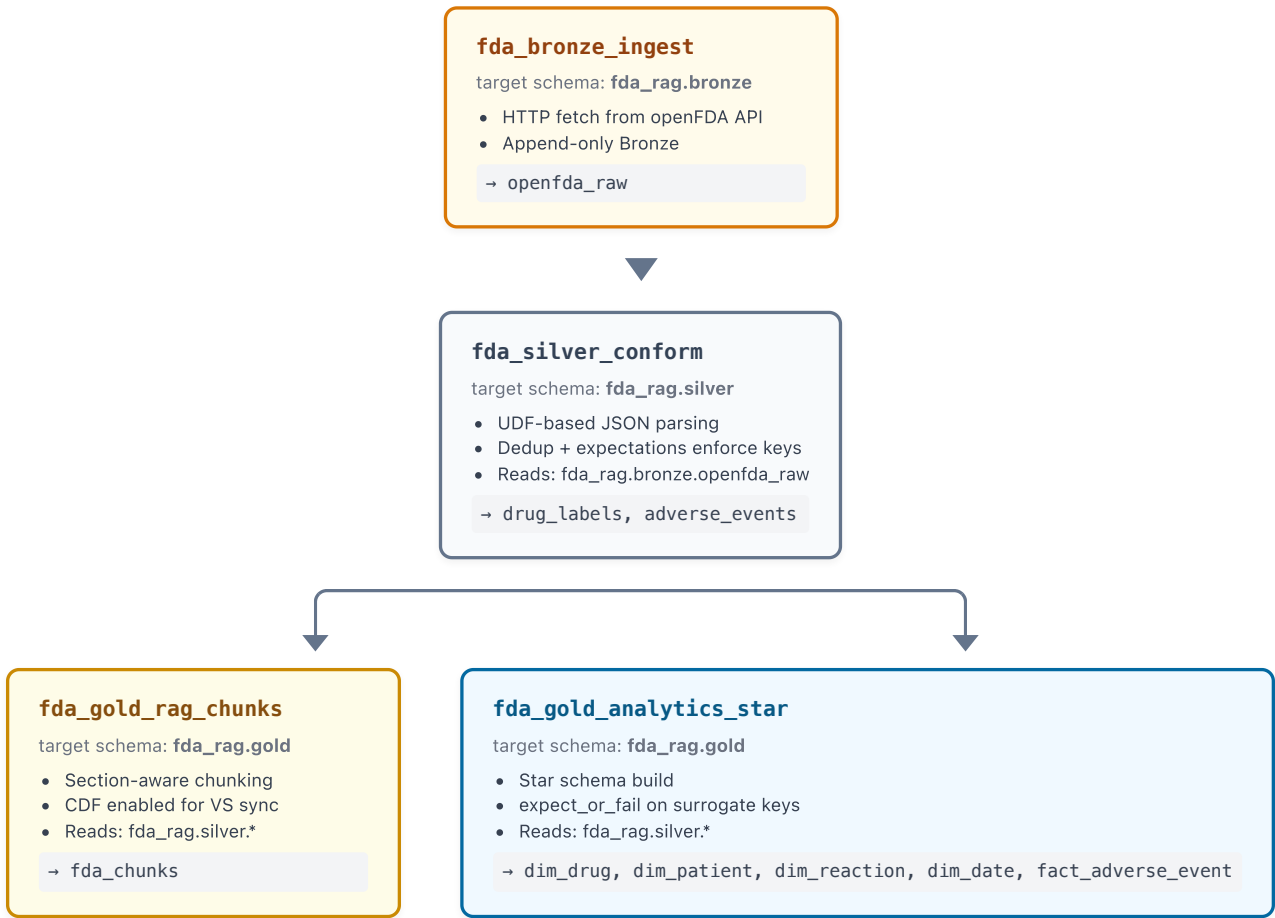
Reading the diagram. Each medallion box shows two implementation paths: the original imperative notebook (**notebook**) and the declarative pipeline equivalent (**DLT**). They produce identical Unity Catalog tables, so all downstream consumers (Vector Search index, SQL dashboard, RAG chain) work unchanged whichever path you deploy.

Key design decision: Both Gold layers derive from one Silver layer, so business definitions live in exactly one place. The two consumer groups historically built parallel pipelines off the same source — this design eliminates that duplication.

Deployment Topology — 4 DLT Pipelines Orchestrated by a Workflow



Each medallion layer is its own pipeline so layers can have independent SLAs, owners, and schedules. Gold pipelines run in parallel after Silver completes. Cross-pipeline reads use `spark.read.table("fda_rag.<schema>.<table>")` because `dlt.read()` only sees tables in the current pipeline.



Wrap all four in a single **Databricks Workflow** with task dependencies: `bronze → silver → [gold_rag, gold_analytics]`

Notebooks vs Declarative Pipelines — What Each Adds

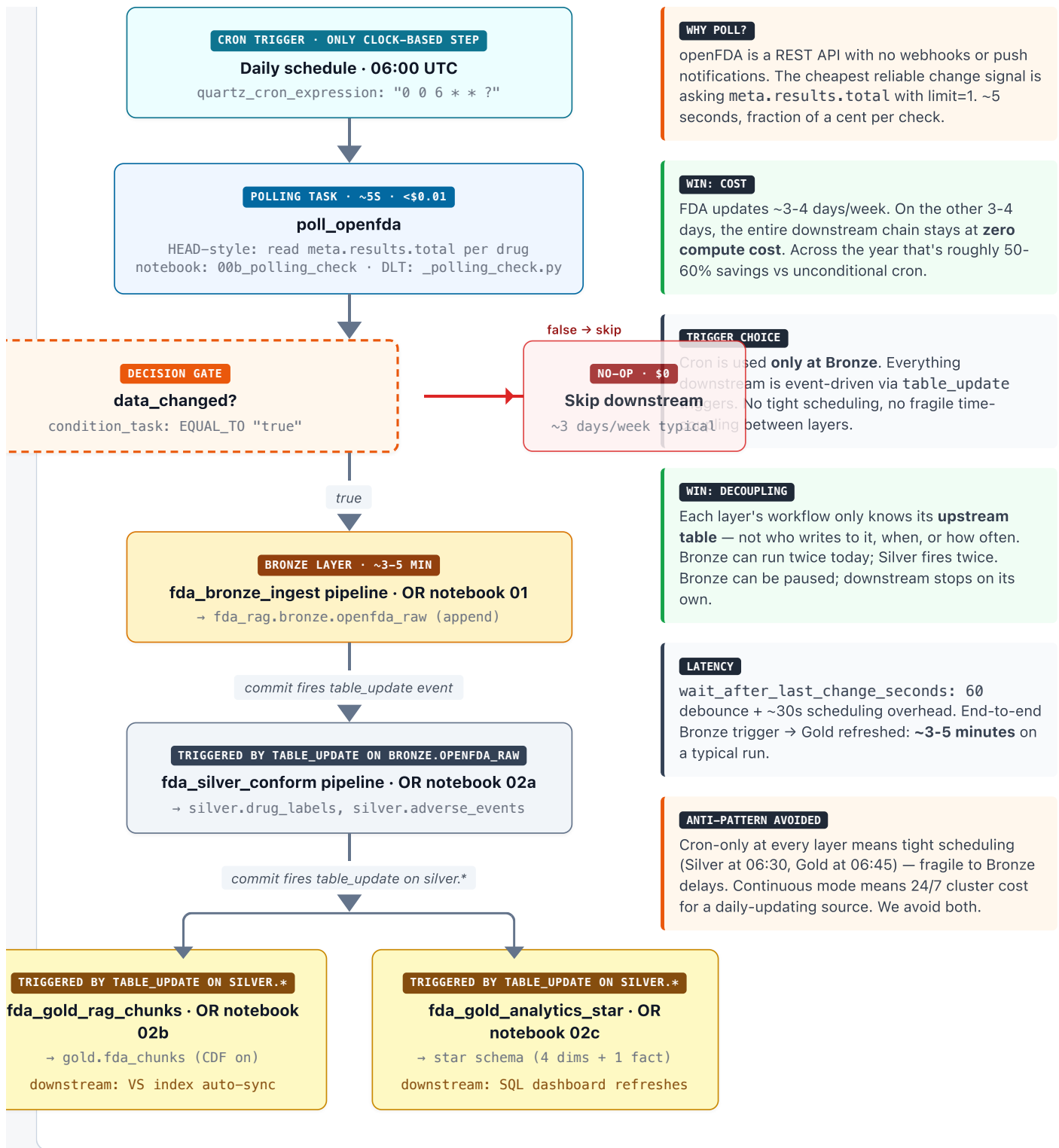
Capability	Notebooks imperative	DLT declarative
Data quality	Implicit – null filters in code	<code>@dlt.expect_or_drop</code> / <code>_or_fail</code> with built-in DQ dashboard
Lineage	Manual – derived from run history	Auto-generated in Unity Catalog from <code>dlt.read()</code> graph
Incremental processing	Manual MERGE / watermark logic	Native – materialized views detect changes
Schema evolution	<code>overwriteSchema=true</code> or fail	Default safe evolution + alerts
Backfill / full refresh	Re-run cells	UI button per pipeline
Orchestration	Workflow tasks calling notebooks	Workflow tasks calling pipelines · DAG within each pipeline auto-resolved

Capability	Notebooksimperative	DLTdeclarative
When each fits	Prototyping · ad-hoc · ML/model code	Production ETL · medallion · CDC · DQ-critical workloads

Event-Driven Orchestration — Smart Polling Gate + Table-Update Cascade

openFDA has no webhooks → a cheap polling task acts as the gate. Once Bronze commits, Silver and Gold cascade automatically via Unity Catalog `table_update` triggers. Each layer is decoupled — it knows only its upstream table.





❌ NAÏVE: CRON-ONLY AT EVERY LAYER

- Bronze runs unconditionally → appends duplicate rows on no-change days
- Silver scheduled at 06:30 — fails or runs on stale data if Bronze ran long
- Gold scheduled at 06:45 — fragile to upstream delays
- Always-on cluster cost even on no-change days
- No clean "what changed and why" audit trail

✓ EVENT-DRIVEN: POLL → CASCADE

- Bronze runs only when openFDA totals change → no duplicate rows
- Silver fires within ~1-2 min of Bronze commit (no clock coupling)
- Both Gold layers fire in parallel after Silver commits
- Compute cost ≈ 0 on no-change days (~50-60% annual savings)
- Full audit chain: state table + run history



Governance — Multi-Tenant Row-Level Security & PII Masking

Tenant axis: **drug therapy area**. The cardio team sees cardio drugs; the metabolic team sees metabolic drugs; admins see everything. Implemented with two Unity Catalog primitives: `ROW FILTER` and `COLUMN MASK`.

1 · ACCOUNT GROUPS (UC)

- `admins` — sees all data
- `clinical_full_access` — full PII access
- `tenant_cardio`
- `tenant_metabolic`
- `tenant_psych`
- `tenant_otc`

2 · MAPPING TABLE

```
gold._tenant_drug_map
tenant_group · drug_generic

cardio · warfarin
cardio · atorvastatin
metab · metformin
psych · sertraline
otc · ibuprofen
...
```

- `SELECT` restricted to `admins`

3 · UC FUNCTIONS

Row filter

```
tenant_drug_filter(drug_generic) :
```

- Returns true if user $\in$ `admins`
- OR user $\in$ group that has this drug mapped

Column mask `age_pii_mask(age)` :

- Raw age for `admins` / `clinical_full_access`
- Decade bucket otherwise (60-70 $\rightarrow$ 60)

↓ applied via `ALTER TABLE ...`

Tables protected:

```
silver.drug_labels · silver.adverse_events · gold.fda_chunks · gold.dim_drug
```

```
ALTER TABLE silver.drug_labels SET ROW FILTER fda_rag.gold.tenant_drug_filter ON (drug_generic);
ALTER TABLE silver.adverse_events ALTER COLUMN patient_age SET MASK fda_rag.gold.age_pii_mask;
```

⚠ RAG gotcha: vector indexes don't inherit row filters

The VS index is computed from a snapshot of `gold.fda_chunks`. UC row filters are evaluated at *query* time on the source table — not on the index. The RAG chain must apply tenant filtering at retrieval time as a VS metadata filter: `retriever_kwargs={"filter": "drug_generic IN (...)"} . For regulated isolation (e.g., separate sponsor data), use per-tenant indexes instead.`

Deployment Lifecycle — Asset Bundles (dev → staging → prod)

One YAML manifest declares **every** Databricks resource — pipelines, workflows, registered models, serving endpoints. Same manifest deploys to three environments via target-level variable overrides.

SOURCE (THE REPO)

```
├─ databricks.yml
├─ resources/
│  └─ pipelines.yml
│  └─ workflows.yml
├─ serving.yml
├─ notebooks/
├─ pipelines/
├─ workflows/
```

Variable substitution:
`${var.catalog}` ·
`${var.workload_size}`

```
# Validate YAML — no API calls
databricks bundle validate -t dev

# Deploy resources to target
databricks bundle deploy -t dev

# Run a specific workflow
databricks bundle run \
  fda_workflow_event_driven
-t dev

# Promote
databricks bundle deploy -t staging
```

DEV (DEFAULT)

- Mode: development
- Catalog: `fda_rag_dev`
- Schedules: PAUSED
- Prefix: `[user@email]`
- Run-as: caller

STAGING

- Mode: `production`
- Catalog: `fda_rag_staging`
- Run-as: service principal
- Workload: `Small`

PROD



```
databricks bundle deploy -t prod
# Tear down dev when done
databricks bundle destroy -t dev
```

- Mode: production
- Catalog: fda_rag
- Schedules: UNPAUSED
- Workload: Medium
- Run-as: fda-pharma-prod-sp

What's NOT in the bundle (and why): Vector Search endpoint & index, Unity Catalog catalogs/schemas, and RLS DDL. Asset Bundles don't natively support these resource types yet — they're foundational state managed by one-time setup notebooks (00_setup , 03_vector_index , 08_setup_rls). The pattern: *bundles for repeatable resources, setup notebooks for foundational state.*

Implementation Playbook — End-to-End Walkthrough

The full path from a clean Databricks workspace to a deployed RAG endpoint + analytics dashboard, in 8 phases. Each phase has explicit activities, verification, and the files touched.

~2 hr

END-TO-END

8

PHASES

9

NOTEBOOKS

4

DLT PIPELINES

5

WORKFLOWS

7

DELTA TABLES

1

Workspace Prerequisites

~15 min

ACTIVITIES

- Verify Unity Catalog is enabled in your workspace
- Enable Foundation Model APIs: databricks-gpt-5-5-pro + databricks-gte-large-en
- Verify Vector Search service is enabled (Compute → Vector Search)
- Create a Single-Node cluster on DBR 14.3 LTS ML
- Install libraries: langchain-databricks , databricks-vectorsearch

VERIFY

- Cluster shows Running
- Both libraries show Installed
- Both Foundation Model endpoints visible under Serving

FILES:

UI: Compute

UI: Serving

2

Foundational State

~20 min

ACTIVITIES

- Run 00_setup — creates catalog, schemas, volume
- Create Vector Search endpoint named fda-vs-endpoint via UI (wait ~5 min for "Online")
- (Optional) Configure UC account groups for tenants: admins , clinical_full_access , tenant_cardio , tenant_metabolic , tenant_psych , tenant_otc

VERIFY

- Catalog Explorer shows fda_rag with bronze/silver/gold schemas
- VS endpoint status: Online
- 00_setup prints "available" for both endpoints

FILES:

notebooks/00_setup

UI: Compute → Vector Search

Choose & Deploy ETL Path

~10 min

↑

ACTIVITIES

- **Option A — Imperative:** Import notebooks 01–02c, attach to cluster
- **Option B — Declarative (recommended for prod):** `databricks bundle validate -t dev`, then `databricks bundle deploy -t dev`
- Choose option A for prototyping, B for production

FILES: `notebooks/01-02c` `databricks.yml` CLI: `databricks bundle`

VERIFY

- Notebooks visible in Workspace (A), OR
- 4 DLT pipelines visible under Workflows → Delta Live Tables (B)
- 5 workflows visible under Workflows → Jobs (B)

4

First End-to-End Data Run

~30 min

ACTIVITIES

- Trigger the Bronze workflow manually (polling reports first-run change)
- Bronze runs, commits to `fda_rag.bronze.openfda_raw`
- Silver workflow fires automatically on `table_update` event
- Both Gold workflows fire in parallel after Silver commits

FILES: `workflows/dlt_bronze_workflow.json` `notebooks/00b_polling_check`

VERIFY ROW COUNTS

- `bronze.openfda_raw`: ~1,600
- `silver.drug_labels`: ~40–80
- `silver.adverse_events`: ~700+
- `gold.fda_chunks`: ~400–600
- `gold.fact_adverse_event`: ~3,000+

5

Vector Index Build & Sync

~10 min

ACTIVITIES

- Run `03_vector_index` cell 1 — submits index creation
- Wait 5–10 min — watch VS endpoint UI for "Online"
- Run cell 2 — sanity-check similarity search

FILES: `notebooks/03_vector_index`

VERIFY

- Index `fda_chunks_index` status: `Online`
- Sanity search returns 3 grounded results with drug + section

6

RAG Chain · Register · Eval · Serve

~25 min

ACTIVITIES

- Run `04_rag_chain` — builds chain, logs to MLflow, registers in UC
- Run `06_deploy_serving` — creates endpoint (wait ~10 min for Ready)
- Run `05_evaluate` — runs eval set against live endpoint

FILES: `notebooks/04_rag_chain` `notebooks/05_evaluate` `notebooks/06_deploy_serving`

VERIFY

- `fda_rag.gold.fda_assistant` v1 in UC registry
- Endpoint `fda-rag-endpoint` status: `Ready`
- Faithfulness > 0.85, refusal rate > 90% on out-of-corpus

7

Governance — RLS & PII Mask

~10 min

ACTIVITIES

- Run `08_setup_rls` as a workspace admin
- Add a test user to a `tenant_*` group via account console
- Run cross-tenant verification queries from a tenant user account

VERIFY

- Test user sees only their tenant's drugs in `silver.drug_labels`

- Update RAG chain to apply VS metadata filter at query time

- `patient_age` shows decade buckets to non-clinical users
- Admin still sees all data + raw age

FILES: `notebooks/08_setup_rls` `UI: Account console (groups)`

8

API Gateway & Promote to Prod

~10 min

ACTIVITIES

- Local: `pip install -r api/requirements.txt` → `uvicorn app:app --reload`
- Or Docker: `docker build -t fda-rag-api .` → `docker run ...`
- Test: `curl POST /v1/ask` with API key
- Promote: `databricks bundle deploy -t staging` → `-t prod`

VERIFY

- API returns grounded answer with [drug — section] citation
- API includes disclaimer + 200 response
- Prod workflow schedule: `UNPAUSED`
- Prod runs as service principal (auditable)

FILES: `api/app.py` `api/Dockerfile` `databricks.yml` `CLI: databricks bundle deploy`

✓ **Day-2 operations once deployed:** Bronze polling task runs daily at 06:00 UTC and gates downstream cascade. The full chain typically completes in ~3–5 minutes after Bronze commits. Monitor via the inference table view (`07_monitoring`) for refusal rate, latency, and error rate. Re-run `04` → `05` → `promote` alias whenever you ship a new prompt or change retrieval parameters.

